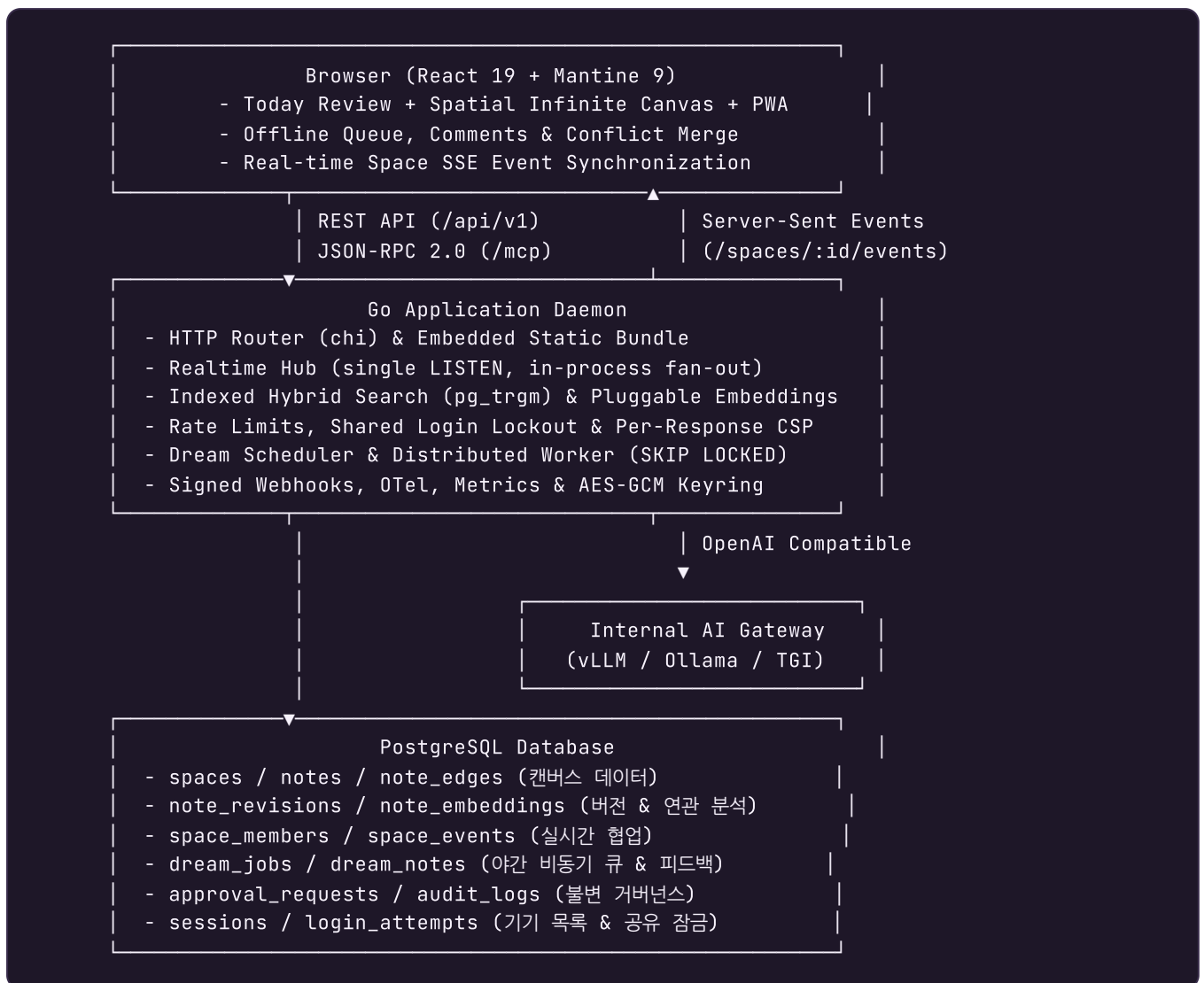


umm 실행 아키텍처 및 불변 조건 (Architecture)

umm은 단일 Go 바이너리와 단일 PostgreSQL 인스턴스만으로 오프라인 폐쇄망에서 무중단 운영이 가능하도록 설계된 **Spatial Thought Memory** 엔진입니다.

1. 시스템 아키텍처 다이어그램



2. 데이터 경계 및 도메인 분리

- **Canvas Domain:** spaces, notes, note_edges
- **Intelligence Domain:** note_revisions, note_embeddings

- **Collaboration Domain:** `space_members`, `space_events`, `notifications`, `note_comments`, `comment_mentions`
- **Identity & Access:** `users`, `sessions`, `oauth_states`, `teams`, `login_attempts`, `ai_quota_reservations`
- **Personalization & Review:** `user_preferences`, `note_reviews`, `api_keys`
- **Governance & Security:** `approval_requests`, `audit_logs`, `app_settings`
- **Dream & LLM Domain:** `dream_jobs`, `dream_notes`, `dream_sources`, `dream_feedback`, `ai_calls`, `ai_eval_cases`, `ai_eval_runs`
- **Automation & Reliability:** `webhook_subscriptions`, `webhook_deliveries`, `idempotency_records`, `product_events`

3. 핵심 아키텍처 원칙

1. 외부 의존성 없는 의미 연관성 분석 (Offline Semantic Engine)

- 연관 생각(Related Thoughts)과 클러스터링은 기본적으로 **192차원 문자 n-gram feature hashing**과 **cosine similarity**로 계산됩니다. 외부 다운로드나 API 호출이 없어 폐쇄망에서 100% 동작합니다.
- v0.8.0부터 관리자가 AI Gateway에 임베딩 모델을 지정하면 OpenAI 호환 `/v1/embeddings`를 대신 사용합니다. 모든 벡터에는 모델명과 정규화된 Gateway endpoint의 SHA-256 fingerprint로 만든 알고리즘 식별자가 함께 저장되고 **같은 알고리즘끼리만 비교**합니다. 모델 또는 Gateway를 바꾸면 같은 모델명을 유지해도 서로 다른 벡터 공간이 뒤섞이지 않고 점진적으로 다시 임베딩됩니다. API key와 timeout은 벡터 공간을 바꾸지 않으므로 fingerprint 대상에서 제외합니다. 원격 응답과 장애 뒤 비교 집합을 로컬로 통일하는 후속 pass를 저장하는 transaction은 모두 작업 시작 시점의 `ai_gateway` 설정 세대로 현재 설정 행을 shared lock 확인하므로, 두 pass 사이 또는 호출 중 설정이 바뀌어도 이전 결과가 새 Gateway의 벡터를 덮어쓸 수 없습니다. 콘텐츠 version이 낮은 벡터도 알고리즘이 다르다는 이유만으로 최신 행을 교체하지 못합니다.
- 게이트웨이 호출이 실패하면 로컬 벡터로 내려가되 저장되는 알고리즘 이름은 로컬로 기록됩니다. 실패한 호출이 성공한 것처럼 남는 경로는 없습니다. 공간으로 범위를 제한한 검색은 접근권한과 `ai_excluded`를 먼저 함께 확인하며, 제외 공간의 검색어도 원격 Gateway로 보내지 않습니다.

1-1. 푸시 기반 실시간 협업 (Realtime Hub)

- `space_events`의 AFTER INSERT 트리거가 `pg_notify`를 호출합니다. PostgreSQL은 알림을 커밋 시점에 전달하므로, 롤백된 변경이 다른 사람에게 보이는 상태는 만들어질 수 없습니다.
- 인스턴스마다 전용 연결 하나가 `LISTEN umm_space_events`를 유지하고, 도착한 알림을 프로세스 안의 구독자에게 팬아웃합니다. 구독자 수와 무관하게 데이터베이스 부하는 일정합니다. 단 `pool_max_conns`가 1 또는 2이면 전용 리스너를 시작하지 않고 기존 1초 안전 풀링을 사용해 request pool의 모든 연결을 readiness·인증·transaction 요청에 남깁니다. 상한 3부터 listener를 시작하면서 request용 두 연결을 보존합니다. 목록과 unread count를 함께 반환하는 알림 endpoint는 count와 rows를 순차 실행해 request 연결 하나만 사용하고, 짧은 Dream 채택 transaction도 같은 연결에서 권한을 확인합니다. 외부 호출 동안 유지하는 access lease는 request pool과 분리해 AI 최대 2개, 웹훅 최대 3개로 각각 제한하므로 인스턴스의 최악 연결 상한은 `pool_max_conns + 5`입니다.

- 알림은 페이로드를 신뢰하지 않습니다. 깨어난 리더는 자신이 마지막으로 보낸 sequence 이후를 다시 조회하므로, 알림이 합쳐지거나 유실되어도 이벤트를 건너뛰지 않습니다.
- 리스너가 끊기면 지수 백오프로 재연결합니다. 연결 상태 전환 자체가 모든 SSE 구독자를 coalesced 신호로 즉시 깨우므로, 리더는 기존 30초 safety-net deadline을 기다리지 않고 cursor를 따라잡은 뒤 타이머를 1초 폴링으로 재설정합니다. 재연결 신호에서는 다시 30초 안전망으로 돌아가며 협업이 멈추는 구간이 없습니다.

1-2. 인덱스를 타는 하이브리드 검색

- 키워드 조건은 `단어마다 하나의 ILIKE` 를 AND로 묶은 형태로 생성됩니다. 각 조건이 `pg_trgm` GIN 인덱스를 타고 PostgreSQL이 비트맵을 교집합합니다.
- 인덱스는 `(title || ' ' || content)` 표현식 위에 만들어집니다. 코드의 표현식과 한 글자라도 달라지면 조용히 순차 스캔으로 되돌아가므로, 두 값이 일치하는지 검사하는 테스트를 함께 둡니다.
- 의미 점수 후보는 최근 문서로 상한을 둡니다. 큰 워크스페이스에서 한 번의 검색이 전체 스캔이 되지 않게 하기 위해서입니다.

2. 낙관적 동시성 제어 (Optimistic Concurrency Control)

- 각 생각 노드는 단조 증가하는 `version` 번호를 가집니다.
- 다중 사용자가 동시에 작업하더라도 버전 충돌을 감지하고 409 Problem Details에 최신 서버 메모를 포함합니다. 브라우저는 내 변경과 서버 변경을 나란히 보여 주고 선택·병합한 뒤 새 버전으로 저장합니다.
- 오프라인 변경은 PWA local queue에 먹통 키와 함께 보관됩니다. 재연결 시 안전한 순서로 replay하고 같은 메모의 연속 PUT은 마지막 상태로 합칩니다. 모든 browser storage 읽기·쓰기·삭제는 예외 경계 안에서 수행하며, quota 또는 권한 오류가 나면 mutation을 저장했다고 보고하지 않습니다. 인증된 queue owner는 메모리에도 유지해 저장소 접근이 차단되어도 앱 초기화와 상태 조회를 계속하고, 읽을 수 없거나 손상된 기존 queue는 빈 배열로 덮어쓰지 않습니다. 인증 복구 가능성이 있는 일반 401/403은 queue를 멈추지만, 공간에서 제거되었거나 생각이 삭제되어 댓글 해결·삭제를 더는 적용할 수 없는 경우 서버가 `comment-mutation-forbidden` 403을 반환하므로 그 항목만 사유를 알리고 제거한 뒤 이후의 독립 변경을 계속 처리합니다. 서비스 워커의 / 앱 셀 캐시는 성공한 `text/html` 탐색 응답만 갱신하므로 manifest·asset JSON·SVG를 주소창에서 직접 열어도 오프라인 셀이 비-HTML 응답으로 바뀌지 않습니다. 8자 content hash가 붙은 `assets/` bundle만 1년 immutable이고, manifest·service worker·아이콘·asset manifest 같은 고정 URL은 `no-cache` 로 매 배포 재검증합니다.

3. 분산 큐 & DB 트랜잭션 (SKIP LOCKED)

- 외부 메시지 브로커(RabbitMQ, Redis 등) 없이 PostgreSQL의 `FOR UPDATE SKIP LOCKED` 를 활용하여 Dream 백그라운드 작업을 분산 워커 간 중복 없이 안전하게 임대(Lease) 처리합니다.
- 생성된 Dream은 `dream_notes.note_id IS NULL` 인 검토 후보로 먼저 저장됩니다. Today·검토함·상세와 source query는 Dream 소유권뿐 아니라 현재 공간 owner/member 권한을 함께 확인하므로 공유가 회수되면 파생 본문·공간명·원본도 응답에서 빠집니다. AI Assist와 자동 Dream 생성은 선택·선별된 원본 note 행, 원본 공간 행과 membership을 잠그며 재생성·발전은 Dream 행과 Dream 공간까지 같은 lease에 포함합니다. 공간 ID를 정렬된 순서로 잠근 채 외부 Gateway 호출 종료까지 유지하므로, 공유 회수가 먼저 확정되면 쿼터 예약을 호출 전에 취소하고 AI lease가 먼저 시작되면 회수가 호출 종료까지 기다립니다. 긴 lease는 request pool을 빌리지 않는 인스턴스당 2-slot PostgreSQL 용량으로 제한되어 세 번째 호출은 연결을 점유하지 않고 대기합니다. 자동 생성에서 품질을 통과한 Dream·source·알림도 이 transaction에서 함께 확정되어 point-in-time 조회 뒤 권한 변경으로 본문이나 부분 후보가 남지 않습니다. 채택 요청은 같은 방식으로 현재 공간·편집 membership을 잠근 뒤 메모와

`dreamed` 연결선을 한 번만 생성합니다. 채택 후 발전 결과도 같은 권한·Dream 행 잠금 아래 새 메모와

`expanded` 연결선을 원자적으로 만들며, 동일 본문 재시도는 기존 결과를 반환합니다.

- Dream 노출·숨김·선호 피드백은 목록 응답 시점이 아니라 브라우저 `IntersectionObserver` 에서 카드가 50% 이상 보인 시점에 기록됩니다. 서버는 Dream 행과 현재 공간 membership을 같은 transaction에서 잠근 뒤 상태와 개인화 점수를 함께 확정하므로 권한 회수 뒤 보관한 ID로 피드백을 만들 수 없습니다. 알림의 `resourceType/resourceId` 는 Dream 카드 또는 공유 공간 딥링크로 해석됩니다.
- 공간·메모의 `ai_excluded` 정책은 Scheduler 자격 계산과 AI Assist·Dream Gateway 직전의 최종 source lease에서 모두 다시 적용되어, 설정 변경 이후의 신규 AI 처리에서 제외됩니다.

4. Daily review와 하이브리드 탐색

- `/today` 는 14일 이상 검토하지 않은 생각, 사용자가 지정한 다시 보기, 연결선이 없는 생각, 대기 Dream, 최근 댓글을 현재 권한과 삭제 상태 범위 안에서 집계합니다. 알림 목록과 unread count도 note 알림의 현재 생각 행과 Dream 알림의 현재 Dream 행을 join해 실제 공간 접근권한을 같은 기준으로 적용하며, legacy 알림의 비어 있는 `resource_space_id` 를 신뢰 경계로 사용하지 않습니다. `note_reviews` 가 공유 메모의 검토·고정 상태를 사용자별로 분리하고, `review_digest` 는 협업 활동 query만 제어해 기본 검토 신호를 제거하지 않습니다.
- 검색은 완전 로컬 192차원 feature-hash cosine 점수와 제목·본문·공간 키워드 점수를 결합합니다. 응답용 본문 일부가 잘려도 전체 본문의 키워드 일치 플래그를 점수에 보존하며, 공간·종류·수정 시각 필터와 opaque cursor를 지원합니다.
- 백링크는 실제 `note_edges` 의 incoming/outgoing 방향을 반환하며 연관 생각은 의미 유사성을 보완합니다.

5. 자동화·관측성·공급망

- 도메인 변경, PostgreSQL SSE log, 활성 구독별 `webhook_deliveries` outbox는 같은 트랜잭션에서 커밋됩니다. 세 워커가 가용 dispatch slot을 확보한 뒤 `FOR UPDATE SKIP LOCKED` 로 대기 또는 lease가 만료된 처리 항목을 선점하므로 프로세스 재시작과 순간 부하 뒤에도 전달을 이어갑니다. 각 워커는 `claimed_at` 세대가 정확히 일치하는 delivery와 subscription·owner·space·membership을 잠그고 실제 HTTP 응답, terminal delivery 전환, payload 삭제, subscription counter 갱신까지 같은 transaction에서 확정합니다. 권한 회수·사용자 비활성화·구독 중지가 먼저 끝나면 외부 전송을 시작하지 않고, 전달 lease가 먼저 시작되면 정책 변경이 그 종료까지 기다리므로 point-in-time 확인 뒤 캡처한 payload가 새 정책을 넘어 나가지 않습니다. HMAC 서명, SSRF 재검증, 제한 재시도와 자동 중지를 적용하며, at-least-once 전달 시도의 중복은 delivery UUID로 수신 측이 제거합니다. terminal metadata는 30일 보존합니다. 외부 오류는 잘못된 UTF-8을 제거하고 byte 상한 안의 완전한 rune 경계에서 잘라 PostgreSQL 상태 전환과 실패 횟수 갱신을 안정적으로 끝냅니다.
- HTTP 계층은 low-cardinality route pattern으로 Prometheus count·latency를 기록합니다. 표준 OTLP 환경변수가 있을 때만 OpenTelemetry exporter를 초기화합니다.
- 태그 파이프라인은 offline image archive와 SPDX SBOM을 만들고 checksum 및 GitHub artifact attestation을 발급합니다.

6. 남용 방지와 수평 확장의 경계

- 로그인 실패 횟수와 AI 하루 사용량은 PostgreSQL에서 셉니다. 비밀번호 확인부터 실패 기록 또는 session commit까지 주소·계정별 advisory transaction lock 안에서 처리하고 모든 DB 작업은 같은 연결을 사용하므로, 인스턴스가 여러 대이거나 pool 상한이 1이어도 병렬 추측이 설정한 실패 상한을 넘어가지 않습니다.

- 일반 API 요청 한도는 인스턴스별 인메모리 토큰 버킷입니다. 요청마다 데이터베이스를 왕복하면 막으려는 부하보다 비용이 커지기 때문이며, 그 결과 실효 상한은 $\text{설정값} \times \text{인스턴스 수}$ 가 됩니다. 전역으로 지켜져야 하는 한도는 앞의 두 가지이고, 그것은 데이터베이스에서 셉니다.
- 보안 섹션의 whole-object PUT은 설정별 PostgreSQL advisory transaction lock을 얻은 뒤 최신 행을 읽고, 구버전 payload가 생략한 다섯 남용 방지 필드만 병합합니다. OIDC `client_secret`과 AI Gateway `api_key`의 마스킹 저장도 같은 방식으로 비밀 필드를 생략해 최신 암호문을 transaction 안에서 병합하며, master-key 회전은 같은 두 lock을 정해진 순서로 잡고 재암호화합니다. 동시 관리자 저장과 롤링 업그레이드·키 회전이 겹쳐도 먼저 확인한 한도나 새 암호문을 stale snapshot으로 지우지 않으며 명시한 `0`은 omission과 구분합니다.
- 계정별 로그인 잠금 임계값은 주소별의 3배입니다. 아이디를 아는 사람이 남의 계정을 임의로 잠글 수 있는 상태를 피하기 위한 의도적인 비대칭입니다.
- 만료된 세션·OAuth state·재시도 기록·로그인 실패 기록은 15분마다 정리됩니다. 모든 인스턴스가 동시에 실행해도 안전합니다.
- `scripts/multi-instance-smoke.sh`가 이 세 가지(교차 인스턴스 이벤트 전달, 교차 인스턴스 맥등 재시도, 공유 로그인 잠금)를 CI에서 실제로 검증합니다.